

Atlas Inventory Workshop Exercises

Introduction

These exercises are meant to introduce agentic coding with the [opencode](#) harness on a small, framework-free Java codebase. The application itself is documented in the [repository README](#). Build it once with `mvn clean verify` before you start.

These exercises are written for `gpt-4.1-mini`, a model which dates back to April 2025. Models evolve fast and by today's standards this is an old and weak one. To successfully code with this model, we must ensure our prompts are explicit and tasks are narrow. At this stage, that is a useful limitation. A weaker model makes your model supervision skills visible, because it will drift if you let it. It is worth noting though that today's frontier models have greatly improved capabilities to what we will be seeing today.

Run order

Work through the reference material first, then the hands-on exercises in order. Later exercises assume the `AGENTS.md` generated in 1.2 exists.

Reference — read these first (~30 minutes total)

	File	What it covers
1	syntax/01-getting-started.md	Install check, first prompt, <code>/init</code> and <code>AGENTS.md</code> , a tour of <code>opencode.json</code>
2	syntax/02-context-modes-and-mentions.md	<code>@</code> mentions, subagents, Build vs Plan mode
3	syntax/03-context-engineering.md	Zero-shot vs few-shot prompting

Hands-on — do these in order (2 hours 15 minutes total)

	Exercise	Time	Skill
1	Read-only code investigation	20 min	Investigate before you change anything
2	Small feature with tests	30 min	Plan → approve → implement → verify
3	Diagnose and fix a defect (trainer setup)	25 min	Reproduce first; check the diagnosis, not just the green test
4	Constrained refactoring	25 min	Hold an agent to explicit invariants
5	Individual mini-capstone	35 min	The whole loop, unaided

Your trainer will have set up a defect in the codebase ahead of exercises 3 and 5.C. See the setup section at the top of each.

The through-line

Every exercise ends with the same question in a different costume: **would you approve this diff without reading it?** The agent is fast and usually right, which is exactly what makes an unreviewed diff dangerous. What we are practising today not the prompting so much as the testing, validating, and reviewing.